IEEE 829-STYLE TEST PLAN & EXECUTION SNAPSHOT

Smart Content Delivery System

Test Plan and Test Execution Report

Test Plan Identifier	TP-SCDS-001
Version	1.0
Project period evidenced by repository	October 25–November 12, 2026
Plan / execution snapshot	November 12, 2026
Project team	Shreyansh Tandon — Tester Supreeth Prahlada — Developer Shyam Sooraj — Developer
Supervisor / approval	

Prepared from the current repository, documented workflow, available automated tests, and checks actually run for this report. Unrun Mininet scenarios and historical measurements are not represented as current passing results.

1. Test Plan Identifier

TP-SCDS-001, Version 1.0. Project: Smart Content Delivery System (Smart CDN).

2. References

• Repository README.md; docs/architecture.md; docs/agile-workflow.md; docs/product-backlog.md; docs/test-plan.md.

• config/settings.md for fixed addresses, ports, and route ports.

• tests/test_pr2_regressions.py and tests/run_scenarios.py.

• IEEE 829 test-plan structure used as a documentation template; this is a project-specific adaptation, not a claim of formal standards certification.

3. Introduction

The system delivers content over raw TCP through a virtual IP. A Ryu OpenFlow controller selects among three content servers using health/load and measured route conditions in dynamic mode; static mode provides a baseline. Mininet supplies the emulated network. This plan defines test scope, acceptance, execution, responsibilities, schedule, and current evidence.

Test objective: establish reliable delivery and checksum integrity, correct server/path decisions and recovery behavior, reproducible workload handling, and trustworthy experiment reporting. Performance claims require fresh, provenance-rich runs; old or unverified artifacts are not current acceptance evidence.

4. Test Items

• TCP content server and client: request/response, byte-offset resume, retries, SHA-256 validation.

• Ryu controller and selector: VIP handling, flow install/remove/move, health and load-aware selection.

• Path and topology state: probes, route health, port-down handling, direct/backup paths, switch reconnection.

• Mininet scenario harness: smoke, six-client, failover, path, congestion, and experiment scenarios.

• Experiment/report pipeline: CSV measurements, summaries, and plots.

• Route-map visualizer and controller telemetry display.

5. Software Risk Issues

• Wrong server/path choice or stale health telemetry can interrupt service.

• Flow cleanup/migration errors may break active TCP streams or strand stale rules.

• Client resume bugs may truncate/duplicate content or produce a checksum mismatch.

• Topology and controller port/address drift can make scenarios misleading or fail to start.

• Concurrent traffic and Mininet timing can make performance results noisy.

• Old experiment data has unclear/obsolete provenance; mixing it with current runs can invalidate conclusions.

• Network integration depends on privileged Linux networking, Mininet, Open vSwitch, and a running Ryu controller.

6. Features to be Tested

• Virtual-IP content retrieval and checksum verification.

• Dynamic selection among healthy servers and static-baseline behavior.

• Heartbeat-based server failure handling and resumable client downloads.

• srv1 direct-path failure, degradation, backup-path migration, and recovery.

• Six-client concurrency; congestion-aware behavior; seeded random workload replay.

• Experiment output validity and report generation from current run data.

• Safe text rendering and route-event reporting in the live route visualizer.

7. Features Not to be Tested

• Internal correctness/security of third-party Ryu, Mininet, Open vSwitch, Python, and plotting libraries.

• Production-scale physical network throughput, hardware limits, and external network infrastructure.

• Formal security certification or exhaustive penetration testing; this project has no login/database/web application corresponding to the unrelated sample template.

• Performance claims based only on files under results/archive/unverified/.

8. Approach

Use a test pyramid and risk-based regression: fast automated unit/regression and syntax checks on every change; Mininet scenario tests for controller/topology behavior; full mode/condition experiment runs when producing reportable performance results. New behavior or defects should have a focused regression test before implementation where practical.

Level	Execution	Acceptance / evidence
Unit / regression	uv run python -m unittest discover -s tests -v	All discovered tests pass; no unexpected errors.
Static / syntax	uv run python -m compileall -q app controller topology tests; git diff --check	Exit 0; no syntax or whitespace errors.
Scenario integration	Run Ryu in dynamic mode; smoke, then relevant scenario with tests/run_scenarios.py.	Expected traffic behavior, valid bytes/checksum, and controller state/log evidence.
Experiments	Run normal/congested/failed conditions in static and dynamic modes; regenerate report.	Configuration, mode, condition, client count, repetitions, and run identity recorded; failed/excluded runs explained.

Scenario commands and prerequisites are in README.md. Start the controller separately; dynamic mode is required for failover/path/adaptive-congestion acceptance. If prerequisites are absent, record BLOCKED/NOT RUN rather than PASS.

9. Item Pass/Fail Criteria

• PASS: observed result meets the listed expected result and has reproducible command/output evidence.

• FAIL: a run completed but violated expected behavior, or an automated assertion failed.

• BLOCKED / NOT RUN: execution did not occur because required environment/service was unavailable. It is excluded from pass-rate calculation.

• Experiment measurements are accepted only with identified mode, condition, configuration, repetitions, and current-code provenance. A passing unit suite does not prove live network behavior.

10. Suspension Criteria and Resumption Requirements

Suspend live scenarios if the controller is unavailable, OVS/Mininet cannot establish a stable topology, host cleanup leaves stale state, test traffic could affect unrelated systems, or a critical checksum/flow failure makes further results unreliable. Preserve logs and partial data, record the blocker, clean up safely, fix the cause, and rerun the failed test plus relevant regression suite before resuming.

11. Test Deliverables

• This PDF test plan and execution snapshot.

• Automated regression suite in tests/.

• Scenario output/logs and current experiment data under results/ when run.

• Issue/backlog updates and follow-up evidence in docs/product-backlog.md and docs/agile-workflow.md.

12. Remaining Test Tasks

• Run adaptive-congestion and seeded random-congestion scenarios after interactive sudo authorization is available.

• Run quick or full experiment matrices in dynamic and static modes with recorded run configuration; regenerate reports from the resulting current data.

• Expand automated tests around selector health/cost boundaries and client resume/checksum error cases.

• Verify clean-checkout setup in documented Ubuntu and record actual blockers and versions.

13. Environmental Needs

Target: Ubuntu/Linux, Python environment pinned by pyproject.toml/uv.lock (Python 3.9), Mininet, Open vSwitch, compatible Ryu, and sudo privileges for network namespaces. README also requires iperf for congestion scenarios.

Observed host on November 12, 2026: Linux; Mininet command present (reported 2.3.1b4). OVS daemon processes and database were available after phone-approved sudo; the openvswitch-switch unit itself is not installed on this host. The Ryu controller listened on ports 6653 and 9000 during scenario execution.

14. Staffing and Training Needs

Team needs familiarity with Python 3, TCP client/server behavior, OpenFlow/Ryu, Mininet/OVS, Linux networking, checksums, and experiment variance. Developers and tester should review README and architecture docs before live scenario execution.

15. Responsibilities

Name	Role	Responsibility in this plan
Shreyansh Tandon	Tester	Test-case execution, evidence capture, and result status reporting.
Supreeth Prahlada	Developer	System implementation, defect fixes, and regression support.
Shyam Sooraj	Developer	System implementation, defect fixes, and regression support.

16. Schedule

Repository history begins October 25, 2026; the current test-plan and execution snapshot is November 12, 2026. Historical commit dates establish the project window only; they do not prove that a formal test phase occurred on a particular day.

Phase / task	Schedule	Status and evidence
Repository project work	From Oct 25, 2026	Repository history begins; not a test completion claim.
Test plan and case mapping	Nov 12, 2026 snapshot	Prepared for this report.
Automated regression and static checks	Nov 12, 2026	Executed; all passed.
Basic dynamic integration	Nov 12, 2026	Smoke, six-client, failover, path failure and degradation scenarios passed.
Congestion and fresh experiment matrix	After sudo authorization; date TBD	Blocked/not run; report regenerated from existing 180-row CSV only.

17. Planning Risks and Contingencies

• **OVS/controller unavailable:** do not start/count scenario results; restore service and verify controller ports 6653/9000 before rerun.

• **Stale Mininet state:** use documented cleanup before a controlled run; capture diagnostics before cleanup if investigating a failure.

• **Unstable timing/noisy load:** repeat runs, retain all attempts, and document seed/configuration; do not silently discard outliers.

• **Data provenance gap:** keep older/unverified artifacts separate and regenerate reports from current runs only.

• **Failure affects multiple tests:** suspend dependent scenarios, file a defect/backlog item, fix, then rerun focused cases and regressions.

18. Test Case Specification and Execution Results

Execution snapshot: November 12, 2026. Results below are from commands run against this checkout. Cases that could not run because privileged phone authorization timed out are explicitly marked NOT RUN.

ID	Test / procedure	Expected result	Actual result / evidence	Status
TC01	Reject adaptive-congestion with static mode.	Argument parser exits with usage error (code 2).	Automated regression passed.	PASS
TC02	Seeded congestion choice is replayable for seeds 42–45.	Same expected sequence on repeated evaluation.	Automated regression passed.	PASS
TC03	Flow move publishes latest route event.	Event includes timestamp, client, server, and new route.	Automated regression passed.	PASS
TC04	Switch reconnect clears only that switch's stale down ports.	Other switches' down-port state remains.	Automated regression passed.	PASS
TC05	Route visualizer handles route name as text, not HTML.	Uses textContent; unsafe innerHTML pattern absent.	Automated regression passed.	PASS
TC06	Compile app, controller, topology, and test Python modules.	Compile command exits successfully.	uv run python -m compileall -q app controller topology tests exited 0.	PASS
TC07	Check working diff whitespace/errors.	git diff --check exits successfully.	Command exited 0.	PASS
TC08	Standalone TCP client/server loopback transfer with checksum verification.	Client receives complete payload and reports checksum=OK.	PASS: 1,048,608 bytes transferred; SHA-256 7efe7a6c9a0f905fabcd49e82b45b054869ceb309d6d46788c491a62cd1ec73b; retries=0.	PASS
TC09	Dynamic Mininet smoke: download via VIP and verify checksum / flow rules.	Transfer completes and checksum matches; expected CDN flows present.	PASS: c1 and c2 each downloaded 2,097,152 bytes with checksum=OK and 0 retries; s1 forward/reverse CDN rules were installed. Mininet HTB quantum warnings appeared as documented.	PASS
TC10	Six-client concurrent smoke.	c1–c6 complete downloads with valid checksums.	PASS: c1–c6 all reported ok; 0 retries each; runner accepts only checksum=OK results.	PASS
TC11	Server-failure resume.	Client resumes after server loss and final checksum matches.	PASS: c1 received 104,857,600 bytes, checksum=OK, retries=1; srv1 restarted afterward.	PASS
TC12	srv1 direct-path failure and backup recovery.	Live flow moves to backup without reconnect; new flow returns direct after recovery.	PASS: 100 MiB active transfer moved from s1 port 5 to port 12, checksum=OK/retries=0; a new transfer used port 5 after recovery.	PASS
TC13	Path-degradation migration.	Existing flow moves to backup under degradation and remains checksum-valid.	PASS: 100 MiB active transfer moved to s1 port 12, checksum=OK/retries=0; a new transfer used port 5 after recovery.	PASS
TC14	Live adaptive and seeded-random congestion scenarios.	Expected route/server response under load; workload is reproducible.	NOT RUN: adaptive-congestion could not elevate through sudo; the interactive phone authorization request timed out. Randomized live scenario was consequently not run.	NOT RUN
TC15	Quick static/dynamic experiment matrix.	Normal, congested, and failed conditions produce current identifiable measurement rows in both modes.	NOT RUN: Mininet experiment runner requires sudo; interactive phone authorization was unavailable. No new rows were added to experiments.csv.	NOT RUN
TC16	Generate summary and plots from the experiment CSV.	Report command exits successfully; summary and figures are generated from available data.	PASS: report command exited 0 and regenerated summary/figures from the existing 180 data rows. This is not a fresh experiment result.	PASS

Status key: PASS = executed and met expected result; NOT RUN = pending/unverified, excluded from pass-rate denominator.

19. Test Execution Summary

Measure	Result
Test cases designed in this plan	16
Cases executed	14
Cases passed	14
Cases failed	0
Cases not run / blocked	2
Pass rate among executed cases	100% (14/14); excludes blocked congestion and fresh experiment cases
Unit tests	5 passed, 0 failed
Standalone TCP integration	1,048,608 bytes; checksum verified; 0 retries
Mininet scenarios	Smoke, six-client, failover, path failure and path degradation passed
Report generation	Passed using existing 180-row data file; no new experiment rows
Fresh live performance results	None collected for this snapshot

Interpretation: All executed unit, static, standalone TCP, Mininet smoke, six-client, failover, path failure/degradation, and report-generation checks passed. Two test groups remain blocked: live congestion/adaptive routing and fresh dynamic/static experiment runs. The system is not declared fully acceptance-tested or deployment-ready.

20. Conclusion

On November 12, 2026, all five discovered unit tests, both static checks, standalone TCP delivery, dynamic Mininet smoke, six-client smoke, server failover, path failure/recovery, path degradation/recovery, and report generation passed. The path tests preserved the TCP stream and checksums; the server-failure test resumed with one retry. Live adaptive/random congestion and fresh static/dynamic experiment runs remain unverified because sudo phone authorization timed out. The regenerated summary and plots use only the pre-existing 180-row CSV; they are not new performance results. Complete the blocked cases before making a full acceptance or performance claim.

21. Approvals

Name	Role	Signature / date
Shreyansh Tandon	Tester	
Supreeth Prahlada	Developer	
Shyam Sooraj	Developer	
	Supervisor	